

UNIT 2

Image Processing & Feature Extraction

Topics covered:

Edge Detection — Sobel & Canny
Corner Detection — Harris & Shi-Tomasi
Blob Detection — LoG & DoG
Feature Descriptors — SIFT, SURF & ORB
Texture Analysis & Feature Representation

Prerequisite: Unit 1 — image formation, sampling & quantization, color models, basic filtering
Code examples use Python, OpenCV (cv2), NumPy, Matplotlib and scikit-image

Environment Setup

All practicals in this unit assume the following Python environment. Run this once before working through the examples.

```
pip install opencv-contrib-python numpy matplotlib scikit-image

# Quick sanity check
python -c "import cv2; print(cv2.__version__)"
```

Use opencv-contrib-python (not plain opencv-python) — SIFT/SURF live in the contrib modules. SURF is patented and may be disabled in some builds; ORB and SIFT are always free to use.

Common imports used throughout this document:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

def show(img, title=''):
    plt.figure(figsize=(6,6))
    cmap = 'gray' if len(img.shape) == 2 else None
    plt.imshow(img, cmap=cmap)
    plt.title(title); plt.axis('off'); plt.show()

img = cv2.imread('sample.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

Edge Detection

What exactly is an edge?

- Edges are pixels where image intensity changes sharply — they usually mark object boundaries, surface orientation changes, or texture.
- In a real image, edges usually occur where there is a significant change in:
 - brightness/intensity
 - color
 - texture

Edge detection is almost always the first feature-extraction step in a computer vision pipeline, since edges compress an image into a much smaller, semantically useful representation.

Why Edges Matter

Suppose there is an image of cup but a computer doesn't naturally see "this is a cup."

It sees something more like:

120 121 119 118 120 ...

122 121 120 119 121 ...

125 124 123 122 120 ...

...

Thousands or millions of numbers.

But the **shape of the cup** is represented strongly by its boundaries.

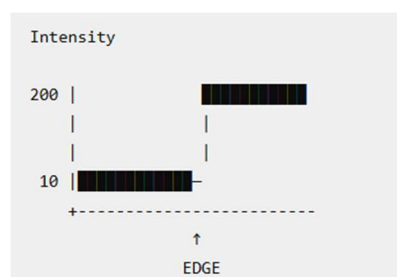
So edge detection converts: **Image** → **important boundaries**

- Boundaries between objects/background carry most of the shape information.
 - Reduces data volume from millions of pixels to a sparse set of edge pixels.
 - Basis for higher-level tasks: segmentation, object detection, shape matching.
-

The basic idea behind Edge Detection

Consider this 1D image:

10 10 10 10 200 200 200 200



Now calculate the change in Intensity (Derivative of Intensity wrt distance)

Intensity of pixels of image: 10 10 10 10 200 200 200

Change in intensity(Derivative): 0 0 0 190 0 0

The derivative is approximately zero where the image doesn't change.

But at the boundary: $200 - 10 = 190 \rightarrow$ derivative becomes large.

- Constant region $\rightarrow$ small derivative
- Edge $\rightarrow$ large derivative

This is the fundamental principle of most classical edge detection.

For an actual image, we don't only move left $\rightarrow$ right.

We can move:

- horizontally
- vertically

Therefore, we calculate derivatives in two directions.

Horizontal derivative

$$G_x = \frac{\partial I}{\partial x}$$

Vertical derivative

$$G_y = \frac{\partial I}{\partial y}$$

Together:

$$\nabla I = \begin{bmatrix} G_x \\ G_y \end{bmatrix}$$

This is called the **gradient**.

The gradient tells us:

1. How strong the change is
2. Which direction the change is happening

At every pixel, gradient magnitude and direction are computed as:

- **Magnitude** (G) = $\sqrt{G_x^2 + G_y^2} \rightarrow$ It tells how strong is the edge at this location?
- Sometimes we use the faster approximation: $G = |G_x| + |G_y|$
- **Gradient Direction** ($\theta = \tan^{-1} \left(\frac{G_y}{G_x} \right) \rightarrow$ This tells us the direction of maximum intensity change.

How do we calculate these derivatives on an image?

- In filtering we used a small matrix (called kernel) and slide it over the image. For edge detection, we use special kernels designed to detect changes.
- The two most famous ones are: **Sobel** and **Prewitt**
- There are also: **Roberts** **Laplacian** and **Canny**

Gradient-Based Edge Detection — Sobel

The Sobel operator approximates the image gradient using two 3×3 convolution kernels —

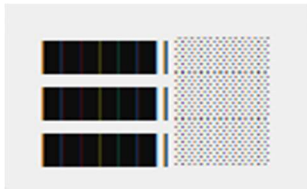
- one for horizontal changes (G_x) and one for vertical changes (G_y).

The kernels are:

Sobel X: $G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$

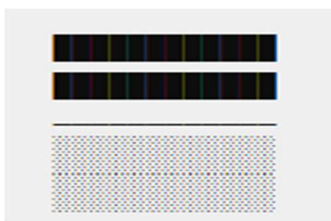
Sobel Y: $G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$

- Sobel X ($\frac{\partial I}{\partial x}$) → compares the left side with the right side (i.e detects **change along x**.)
- So it detects: **Changes in the horizontal direction.**
- Consequently, it is especially strong around **vertical edges**.



Sobel X detects this very well.

- Similarly, Sobel Y ($\frac{\partial I}{\partial y}$) compares the upper region with the lower region (i.e detects **change along y**.)
- Therefore, it detects changes in the vertical direction.
- It is particularly strong around **horizontal edges**.



Sobel Y detects this very well.

At every pixel, gradient magnitude and direction are computed as:

```
Magnitude = sqrt(Gx^2 + Gy^2)
Direction = arctan(Gy / Gx)
```

Larger magnitude $\Rightarrow$ stronger edge. Sobel is fast, simple, and reasonably robust to mild noise because the kernel itself performs a small amount of smoothing.

Practical: Sobel in OpenCV

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray = cv2.GaussianBlur(gray, (3,3), 0) # reduce noise first

sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

magnitude = cv2.magnitude(sobel_x, sobel_y)
magnitude = cv2.convertScaleAbs(magnitude) # back to uint8 for display

show(magnitude, 'Sobel Edge Magnitude')
```

Always blur before Sobel/gradient-based methods — raw gradients amplify pixel-level noise heavily.

Original image $\rightarrow$ Gaussian Blur $\rightarrow$ Noise reduction $\rightarrow$ Edge Detection

Because noise can produce false edges.

Gradient-Based Edge Detection — Prewitt Operator

$$\text{Prewitt } X: \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

$$\text{Prewitt } Y: \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

Prewitt is almost similar to Sobel just one difference:

- Sobel gives more weight to the center row/column while Prewitt treats them equally
- Sobel is generally preferred because of this additional smoothing effect.

Canny Edge Detector

Canny is the most widely used edge detector because it produces thin, well-localized, single-pixel-wide edges with far fewer false detections than raw gradient thresholding.

It is a **multi-stage edge detection algorithm and works in four stages**:

Noise reduction — Gaussian smoothing to reduce noise.

Gradient computation — Sobel G_x , $G_y \rightarrow$ magnitude & direction.

Non-maximum suppression

It does not search for edges. It takes the already-detected thick edge and makes it 1-pixel thin.

Hysteresis thresholding —

Now Canny classifies pixels based on gradient strength

So here two thresholds are defined

- High threshold
- Low threshold
- Pixels above the high threshold are strong edges,
- Pixels between the two thresholds are (weak edges)
- Hysteresis keeps weak edges only if connected to a strong edge and remove isolated weak edges.
- Pixels below low threshold are discarded.

Practical: Canny in OpenCV

```
edges = cv2.Canny(gray, threshold1=100, threshold2=200)
show(edges, 'Canny Edges')

# Auto-threshold using the median of pixel intensities (common heuristic)
def auto_canny(image, sigma=0.33):
    v = np.median(image)
    lower = int(max(0, (1.0 - sigma) * v))
    upper = int(min(255, (1.0 + sigma) * v))
    return cv2.Canny(image, lower, upper)

auto_edges = auto_canny(gray)
show(auto_edges, 'Auto-thresholded Canny')
```

Choosing the thresholds

- Ratio of high:low threshold is typically 2:1 or 3:1.
- Low threshold too high $\rightarrow$ broken/missing edges.
- High threshold too low $\rightarrow$ noisy, false edges.

Experiment with these values:

```
edges1 = cv2.Canny(gray, 30, 100)
```

```
edges2 = cv2.Canny(gray, 50, 150)
```

```
edges3 = cv2.Canny(gray, 100, 200)
```

```
edges4 = cv2.Canny(gray, 150, 250)
```

Sobel vs Canny

Aspect	Sobel	Canny
Output	Gradient magnitude image (thick edges)	Thin, binary single-pixel edges
Noise handling	Minimal (only kernel smoothing)	Explicit Gaussian blur + hysteresis
Speed	Very fast	Slower (multi-stage)
Use case	Quick gradient/feature maps, further processing	Final edge maps, contour detection, shape analysis
Tunable parameters	Kernel size	Blur sigma, low/high thresholds

Mini-Exercise

- Load any image and compare `cv2.Sobel`, and `cv2.Canny` outputs side-by-side using `plt.subplot`.
- Vary Canny's threshold pair and observe how edge continuity changes.

Corner Detection

- A corner is a point in an image where the intensity changes sharply in two directions simultaneously (unlike an edge, which only varies strongly in one direction).
- **Corner detection tries to find interesting points in an image where the intensity changes strongly in multiple directions.**

These points are also called:

- Corner points / Interest points / Keypoints, depending on the algorithm/context
-

Flat region → little/no intensity change in both directions.

Edge → large change in one direction, small change in another

Corner → large change in multiple directions

```

. . . . .
. . . . .
. . #####
. . #####
. . #####
. . #####
. . #####

```

Harris Corner Detector

Steps to detect corner using Harris Corner Detection Technique:

- 1) Convert Color Image to GrayScale (Because corner detection primarily needs **intensity changes**.)
 - 2) Calculate image gradients using Sobel (**X gradient I_x** and **Y gradient I_y**).
 - a. If I_x and I_y both zero → Flat Region
 - b. I_x = large and I_y = small → Strong change in one direction (**Vertical edge**)
 - c. I_x = small and I_y = large → Strong change in one direction (**Horizontal edge**)
 - d. I_x = large and I_y = large → Strong change in both direction (**Corner**)
-

- Suppose at one pixel: I_x = 50 I_y = 80 → It doesn't automatically mean it's a corner.
- Because One pixel may be noisy so we need to examine the **neighbourhood** around the pixel.
- That's why Harris considers a small window. → For every pixel in the window, we look at: I_x and I_y and combinations of them.

Harris looks at how the sum of squared differences (SSD) changes when a small window is shifted in any direction.

This is captured by the second-moment matrix (structure tensor):

$$M = \begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix}$$

Where summation are taken over a small window.

$I_x \rightarrow$ horizontal change

$I_y \rightarrow$ vertical change

$I_x I_y \rightarrow$ helps describe how changes in the two directions are related.

Squaring allows us to measure the **strength of variation**, regardless of whether the intensity is increasing or decreasing.

The matrix M summarizes how much image intensity changes in different directions inside the window.

- 3) Harris then calculates a value called the **Harris response** from the eigenvalues (λ_1, λ_2) of M without explicitly computing them:

$$R = \det(M) - k(\text{trace}(M))^2$$

$$= (\lambda_1 \lambda_2) - k(\lambda_1 + \lambda_2)^2$$

k is a small constant, commonly around: 0.04–0.06

Conceptual explanation of Harris.

- Both λ_1, λ_2 large $\rightarrow$ R large positive $\rightarrow$ corner.
- One large, one small $\rightarrow$ R negative $\rightarrow$ edge.
- Both small $\rightarrow$ $R \approx 0 \rightarrow$ flat region.

Thresholding in Harris

Here we implement some threshold value to achieve: don't want every tiny intensity variation to become a corner.

If Harris Response > Some threshold value $\rightarrow$ Then keep it or discard them

Practical: Harris in OpenCV

```
gray_f = np.float32(gray)
harris = cv2.cornerHarris(gray_f, blockSize=2, ksize=3, k=0.04)
harris = cv2.dilate(harris, None) # enlarge marks for visibility

out = img.copy()
out[harris > 0.01 * harris.max()] = [0, 0, 255] # mark corners in red
show(cv2.cvtColor(out, cv2.COLOR_BGR2RGB), 'Harris Corners')
```

Limitations of Harris:

- Different threshold values can produce different numbers of corners.
- The detector's behaviour depends on the window size. $\rightarrow$ A tiny corner and a large-scale corner may not behave identically.
- Noise creates unwanted intensity variations which can lead to false detection of corners.

Shi-Tomasi (Good Features to Track)

Shi-Tomasi modifies Harris's scoring function $R = \det(M) - k(\text{trace}(M))^2$

Instead of the combined determinant/trace formula, it directly uses the smaller eigenvalue:

$$R = \min(\lambda_1, \lambda_2)$$

A corner is accepted only if this minimum eigenvalue exceeds a threshold — a simpler and often more reliable criterion, which is why this method is the default keypoint selector behind OpenCV's optical-flow tracking pipeline.

Practical: Shi-Tomasi in OpenCV

```
corners = cv2.goodFeaturesToTrack(
    gray, maxCorners=100, qualityLevel=0.01, minDistance=10)
corners = np.int0(corners)

out = img.copy()
for c in corners:
    x, y = c.ravel()
    cv2.circle(out, (x, y), 4, (0, 255, 0), -1)
show(cv2.cvtColor(out, cv2.COLOR_BGR2RGB), 'Shi-Tomasi Corners')
```

Harris vs Shi-Tomasi

Aspect	Harris	Shi-Tomasi
Scoring	$\det(M) - k \cdot \text{trace}(M)^2$	$\min(\lambda_1, \lambda_2)$
Extra parameter	Empirical k (0.04–0.06)	None — more principled
Corner quality control	Threshold on R	<code>maxCorners</code> , <code>qualityLevel</code> , <code>minDistance</code> give direct control
Typical use	Classic corner detection, calibration	Feature points for KLT tracking / optical flow

Blob Detection

A blob is a region that differs in brightness or color from its surroundings and is roughly uniform inside - think of a dark circular spot on a light background.

Edge detection

Sobel/Canny basically ask: **“Where is there a sudden change in intensity?”**

For example:

```
WHITE WHITE WHITE
WHITE WHITE WHITE
-----
BLACK BLACK BLACK
BLACK BLACK BLACK
```

Corner detection

Harris/Shi-Tomasi ask:

“Where does the image intensity change significantly in more than one direction?”

So they detect special points such as:

```

|
|
-----+-----
|
|
```

The + is a corner.

Blob detection

Blob detection asks something different:

“Is there a region here that is significantly different from its surroundings?”

- Edge detection → boundary
- Corner detection → special point
- Blob detection → region/spot

So, the pixels inside the region have some common property that distinguishes them from the surrounding pixels.

That property might be:

- brightness
- darkness
- intensity
- color
- scale
- local contrast

Example:

```

10 10 10 10 10
10 10 [80 80] 10
10 10 [80 80] 10
10 10 10 10 10
10 10 10 10 10

```

Here 80 region is different from the surrounding 10.

Therefore, it can be considered a **blob**.

Laplacian of Gaussian (LoG) to detect Blob

- LoG first smooths the image with a Gaussian of standard deviation σ (Gaussian blur), then
- applies the Laplacian operator (second derivative).

The **Laplacian** is a second-order derivative operator.

- For an image $I(x, y)$

$$\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}$$

Suppose intensity looks like: **Dark** → **Dark** → **Bright** → **Bright**

There is a transition.

First derivative detects the transition.

That's why: **Sobel** → **edge detection**

Second derivative looks at how the rate of change itself changes.

Image → **Gaussian smoothing** → **Laplacian** → **Blob detection**

Blobs appear as strong local maxima/minima of the LoG response.

$$\text{LoG}(x,y) = \nabla^2 [G(x,y,\sigma) * I(x,y)]$$

$\nabla^2 = \partial^2/\partial x^2 + \partial^2/\partial y^2 \rightarrow$ (Laplacian)

G is the Gaussian and

I is the Image

Changing σ changes the scale at which we look at the image.

For small value of σ the image will be blurred less $\rightarrow$ more minute and variation in intensity will be captured $\rightarrow$ small blob detected.

For large value of σ the image will be blurred more $\rightarrow$ less minute and variation in intensity will be captured $\rightarrow$ large blob detected.

Running LoG at multiple values of σ and searching for extrema across (x, y, σ) gives scale-invariant blob detection — the σ at which the response peaks tells you the blob's characteristic size.

The LoG algorithm — simple version

Step 1 — Take image

Step 2 — Apply Gaussian blur (Use different σ values)

A small Gaussian scale is suitable for the small blob and a larger scale is suitable for the large blob. Therefore, different blob sizes require different scales.

Step 3 — Apply Laplacian for every blurred image.

Gaussian $\sigma=1 \rightarrow$ Laplacian

Gaussian $\sigma=2 \rightarrow$ Laplacian

Gaussian $\sigma=3 \rightarrow$ Laplacian

...

Step 4 — Find pixels/regions where LoG response is strong.

Step 5 — Find local maxima

Step 6 — Mark the blob

LoG is mathematically clean but computationally expensive because it needs a fresh convolution at every scale.

i.e. We need to perform Gaussian + Laplacian operations many times, which is computationally expensive.

*So researchers use an approximation which leads us to **DoG — Difference of Gaussians***

Difference of Gaussians (DoG)

DoG approximates LoG far more cheaply: subtract two Gaussian-blurred versions of the image at nearby scales.

$$\text{DoG}(x, y, \sigma) = G(x, y, k\sigma) * I - G(x, y, \sigma) * I$$

Gaussian with $\sigma_1 \rightarrow G_1$

Gaussian with $\sigma_2 \rightarrow G_2$

$G_2 - G_1 \rightarrow \text{DoG}$

Why subtract two Gaussian images?

Suppose we have:

Image A = lightly blurred

Image B = more heavily blurred

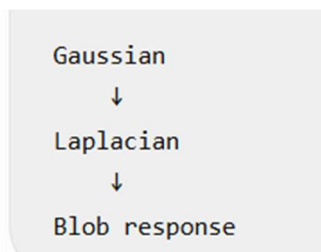
Then: Image B - Image A

highlights structure whose appearance changes significantly between those two scales.

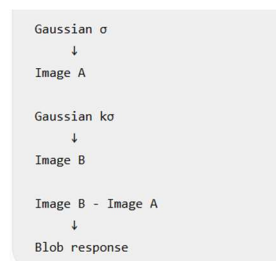
Those structures can correspond to blobs.

DoG detects structures by looking at how the image changes between two nearby scales.

LoG



DoG



Practical: LoG & DoG with skimage-image

```

from skimage.feature import blob_log, blob_dog, blob_doh
from skimage.color import rgb2gray
import matplotlib.pyplot as plt

image_gray = rgb2gray(plt.imread('sample.jpg'))

blobs_log = blob_log(image_gray, max_sigma=30, num_sigma=10, threshold=.1)
blobs_log[:, 2] = blobs_log[:, 2] * np.sqrt(2)  # radius = sigma*sqrt(2)

blobs_dog = blob_dog(image_gray, max_sigma=30, threshold=.1)
  
```

```
blobs_dog[:, 2] = blobs_dog[:, 2] * np.sqrt(2)

fig, ax = plt.subplots()
ax.imshow(image_gray, cmap='gray')
for y, x, r in blobs_log:
    ax.add_patch(plt.Circle((x, y), r, color='red', fill=False, lw=1.5))
plt.title('LoG Blobs'); plt.axis('off'); plt.show()
```

Practical: Simple Blob Detector in OpenCV

```
params = cv2.SimpleBlobDetector_Params()
params.filterByArea = True
params.minArea = 50
params.filterByCircularity = True
params.minCircularity = 0.7

detector = cv2.SimpleBlobDetector_create(params)
keypoints = detector.detect(gray)

out = cv2.drawKeypoints(img, keypoints, None, (0,0,255),
                        cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
show(cv2.cvtColor(out, cv2.COLOR_BGR2RGB), 'Blobs')
```

LoG vs DoG

Feature	LoG	DoG
Full form	Laplacian of Gaussian	Difference of Gaussian
Main idea	Gaussian + Laplacian	Difference between two Gaussian images
Blob detection	Yes	Yes
Computation	More expensive	More efficient
Relationship	Reference method	Approximation of LoG

Feature Descriptors — SIFT, SURF, ORB

Feature Detection vs Feature Description

Imagine you have a photo of a building.

Some places in the image are **not very useful** but some places are very distinctive (a distinctive blob or a **corner of a window**)

These are called **interesting points / keypoints / feature points**.

A **keypoint detector** (Harris, Shi-Tomasi, DoG, etc.) only tells you **WHERE** an interesting point is.

Suppose if I say I found an interesting point at (250, 180). → That's only its location.

Now imagine you take another photograph of the same building but the camera has moved slightly that same corner might now be Image 1: (250, 180) and for this second image: (310, 205) so we need to identify this again that where feature descriptor comes in.



Now I have detected interesting points. Now how do I describe those points so that I can find the same point again in another image? That is exactly where **Feature Descriptors — SIFT, SURF and ORB** come in.

A **keypoint** is a location in the image that is considered interesting.

A descriptor is a numerical representation of the region around the keypoint.

Feature descriptor: WHAT does the area around that interesting point look like?

Simple real-world analogy

Imagine we are trying to identify a person in two photographs.

First we locate: Face → Interesting point = eyes

But simply knowing: “There is an eye at this location” isn't enough.

we need information about the surrounding structure like:

- eye shape
- eyebrow
- nearby edges
- texture
- orientation etc.

That information forms a kind of **signature**.

Feature descriptors do something similar for image regions

What makes a good feature?

Suppose you detect a point in this image

Now the image is:

- moved
- rotated
- resized
- slightly changed in brightness

You ideally want the descriptor to remain similar.

Therefore, a good feature descriptor should ideally be:

- 1. Scale Resistant/Invariant:** Even if image becomes larger/smaller the feature should still be recognizable.
 - 2. Rotation Resistant/Invariant:** Even if image rotates the feature should still match.
 - 3. Illumination Resistant/Invariant:** Even if brightness changes the descriptor should still be reasonably similar.
-

Algorithm Basic idea

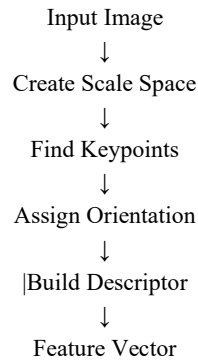
- | | |
|-------------|--|
| SIFT | Detect stable features at different scales and describe local gradients |
| SURF | Similar goal to SIFT but uses faster approximations |
| ORB | Uses FAST keypoints + BRIEF-style binary descriptor, designed to be fast |

SIFT (Scale-Invariant Feature Transform)

SIFT tries to find detect and describe features that remain useful even when the image is:

- resized
- rotated

SIFT(Scale-Invariant Feature Transform) Pipeline



Step 1 — Scale Space

Suppose you have a small object which can appear different at different sizes. If our detector only examines one size, it might miss the feature. Therefore, SIFT looks at the image at **multiple scales**.

Think of scale-space as creating multiple versions of the image.

Scale 1 → slightly blurred
 Scale 2 → more blurred
 Scale 3 → even more blurred
 Scale 4 → heavily blurred

Step 2 : Finding Keypoints in SIFT

Implement DoG and after creating DoG images, SIFT searches for extrema.

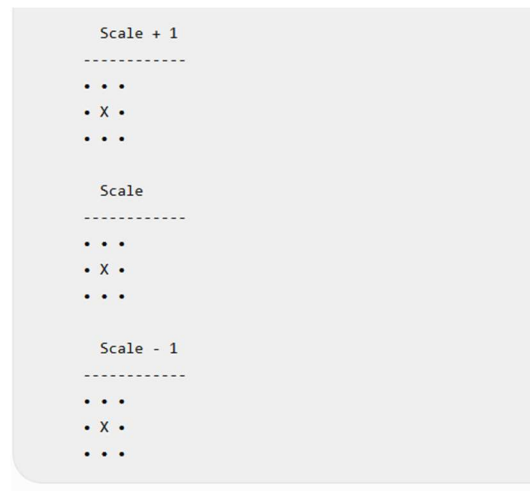
Imagine a pixel (X) surrounded by neighbours:

```

  * * *
  * X *
  * * *
  
```

SIFT compares the point X against surrounding points across scales.

SIFT doesn't only compare neighbours in the same image. It considers neighbouring pixels **across scales as well**.



And if it is a local maximum or minimum, it becomes a candidate key point.

Suppose: $X = 100$ and all its neighbours are: 20, 25, 30, 18, 22... and the corresponding points in the scales above/below are also much smaller. Then: **X is unusually large compared with its surroundings and across scales. → local maxima**

SIFT asks: **"Is this pixel special compared to its neighbours AND is it special at this scale compared with nearby scales?"**

Step 3 — Assign orientation

Now we have a keypoint. But consider:

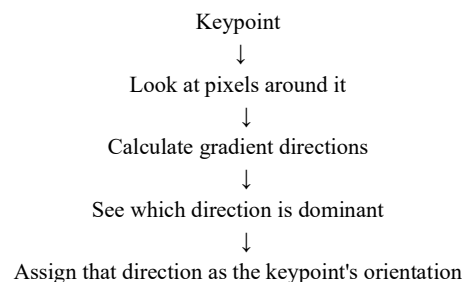


The same feature has rotated. If our descriptor depends directly on the image orientation, matching becomes difficult.

Here SIFT says that **"Before I describe this keypoint, let me figure out which direction this feature is pointing."**

So SIFT examines the pixels around the keypoint and calculates their **gradient directions**(The direction in which the image intensity is **changing strongly**).

Instead of describing the feature using the fixed image coordinates image's absolute orientation, SIFT describes it relative to the keypoint's assigned orientation.



Example demonstration of gradient Histogram

Pixel	Direction	Magnitude	Contribution
P1	20°	5	vote toward 0°–45° bin with weight 5
P2	40°	8	vote toward 0°–45° bin with weight 8
P3	90°	3	vote toward 90° bin with weight 3
P4	10°	1	vote toward 0°–45° bin with weight 1

For point p1 for 45 degree bin $\rightarrow 20/45 * 5 = 2.22$ value to be assigned

And for 0 degree bin it will be: total – 2.22 = 5 – 2.22 = 2.78

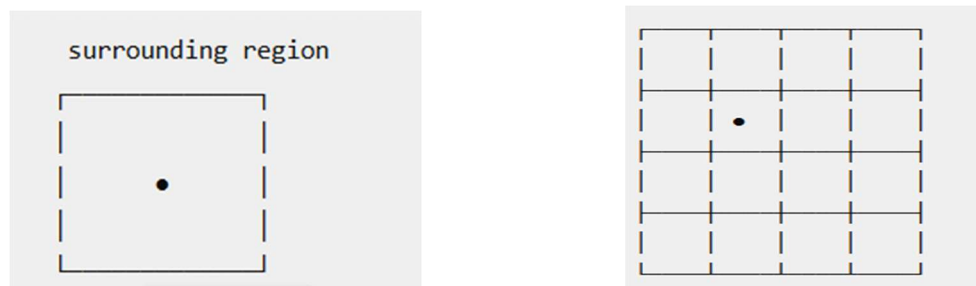
Step 4 — Build the SIFT Descriptor

Now SIFT asks:

"What does the region around this keypoint look like?"

This is the actual **description** part.

SIFT takes a small local region (16 x 16 pixel region) around keypoint and divide the small region in 4*4 grid(cells)



For **each small cell**, we calculate the gradients of the pixels inside it and create an orientation histogram. Typically, SIFT uses **8 orientation bins** (So possible directions are 8 bins)

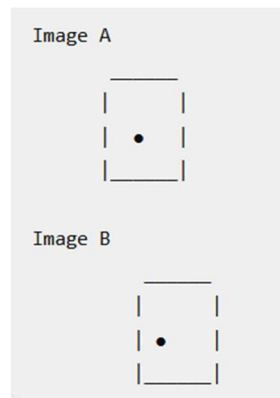
For example, one cell might produce:

0° → 3
 45° → 6
 90° → 2
 135° → 1
 180° → 0
 225° → 1
 270° → 1
 315° → 2

Cell 1 $\rightarrow [3, 6, 2, 1, 0, 1, 1, 2]$

We have 16 cells and each cell gives 8 values $\rightarrow$ 128-dimensional SIFT descriptor

SIFT example



SIFT might detect:

Image A keypoint $\rightarrow (120, 150)$

Image B keypoint $\rightarrow (87, 104)$

Their coordinates are different, but their local structures are similar.

So their descriptors might look conceptually like:

A $\rightarrow [0.12, 0.05, 0.81, \dots]$

B $\rightarrow [0.11, 0.06, 0.79, \dots]$ Almost similar

Practical: SIFT in OpenCV

```
sift = cv2.SIFT_create()
keypoints, descriptors = sift.detectAndCompute(gray, None)

print('Number of keypoints:', len(keypoints))
print('Descriptor shape:', descriptors.shape)    # (N, 128)

out = cv2.drawKeypoints(
    img, keypoints, None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
show(cv2.cvtColor(out, cv2.COLOR_BGR2RGB), 'SIFT Keypoints')
```

SURF (Speeded-Up Robust Features)

SIFT is powerful, but computationally expensive → SURF was designed to achieve similar goals with faster computation.

The key idea behind the speed improvement is the use of: **Integral Images** and **Box Filters**

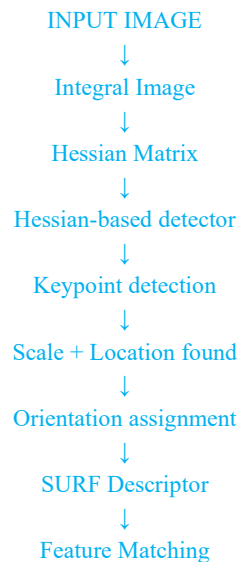
SURF also do the same thing like SIFT:

1. Keypoint detection → Where are the important points?

2. Feature description → What does the area around each important point look like?

But **SIFT** uses: DoG + gradient histograms and SURF uses: Hessian-based detection + Haar-wavelet responses

The basic SURF pipeline



Step 1 — Integral Image

This is one of the most important ideas in SURF.

Suppose we have this tiny image:

```

1 2 3
4 5 6
7 8 9
  
```

The integral image stores cumulative sums.

```

1 3 6
5 12 21
12 27 45
  
```

Integral Image makes rectangular region calculations **very fast**.

For Example we want the sum of all pixels inside a rectangular region → Normally we'd have to add every pixel.

But using an integral image, we can calculate the sum using only **four values**.

SURF uses **rectangular/box filters** repeatedly. Integral images let it calculate the response of those rectangular regions very quickly.

Step -2 Hessian Matrix

In Surf Hessian matrix helps identify regions where image intensity changes strongly in different directions.

A simplified Hessian matrix is:

$$H = \begin{bmatrix} I_{xx} & I_{xy} \\ I_{xy} & I_{yy} \end{bmatrix} \quad \text{Hessian Response} = \det(H) = I_{xx}I_{yy} - I_{xy}^2$$

where:

- I_{xx} = second derivative in x
- I_{yy} = second derivative in y
- I_{xy} = mixed derivative
- Second-order derivatives help identify these blob-like structures.
- **A blob Structure is something like this :** 10 10 100 100 100 10 10
- Intensity rises and then falls. This rise and fall produces a strong second-order response.

SURF uses the Hessian matrix/determinant to detect distinctive points across scales i.e. in SURF instead of looking at the image at only one resolution, we look for features at different scales which makes this scale invariant.

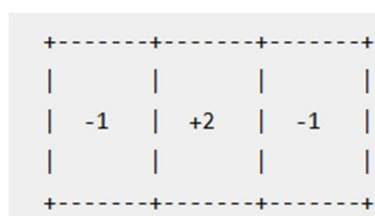
So based on the Hessian Response we now detect the keypoints.

Box Filters

- Suppose we have a Original Image now we need second-order derivatives to form **Hessian Matrix** which is used to detect keypoint.
- But direct computation using Gaussian second-order derivatives is expensive that's why **SURF** uses an approximation called: **Box Filters**.
- Box Filters uses constructed Integral Image rather than original image which leads to box-filter calculations very fast.
- As a result of Box Filters, I got approximate Hessian response which is used to detect **SURF keypoints**.

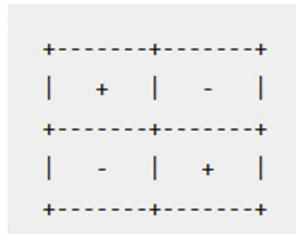
What is a Box Filter?

- A box filter is basically a collection of rectangular regions with different weights.
- For example, imagine D_{xx}



- This filter is designed to behave some what like a second derivative.
- Because it compares: left region $\rightarrow$ center region $\rightarrow$ right region
- If the center is very different from its neighbors, the response becomes large.

D_{xy} $\rightarrow$ For the mixed derivative, SURF uses a pattern involving four rectangular regions, conceptually like:



Let's make a simple 1D analogy/ Example:

Suppose the intensity values are: 10 10 100 10 10

Think of a simple second derivative filter: $[1 \ -2 \ 1]$

Apply it to the center: 10 100 10

We calculate: $1(10) - 2(100) + 1(10) = -180 \rightarrow$ The magnitude is very large: $|-180| = 180 \rightarrow$ Something distinctive is happening here.

In box filters we are summing the pixels values right.

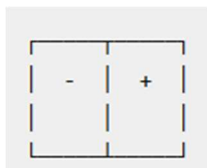
In 2D suppose we have three region and each region consists of 100×100 pixels i.e 10000 pixels

So summing these values make it tough that's where Integral Image comes in picture.

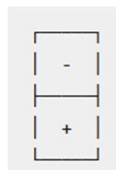
Step 3 — Orientation Assignment using Haar wavelets in SURF

- SURF also determines: **In which direction is the image structure around this keypoint mainly oriented?. This process is called Orientation Assignment.**
- But SURF looks at Haar wavelet responses around the keypoint to assign the Orientation rather than using gradient-direction histogram as SIFT.

Around the keypoint Surf calculates **Haar wavelet responses** in:



x direction $\rightarrow d_x$



y direction $\rightarrow d_y$

- Then it looks for the **dominant direction** of these responses.

```

For each 60° window:
↓
Select the dx, dy responses whose orientations fall inside the window
↓
 $S_x = \sum dx$  and  $S_y = \sum dy$ 
↓
 $R = \sqrt{(S_x^2 + S_y^2)}$ 
↓
Move window by 10°
↓
Repeat
↓
Find maximum R
↓
Take  $S_x, S_y$  of that maximum-R window
↓
 $\tan\theta = (S_y / S_x)$ 
↓
FINAL ORIENTATION

```

Step 4 — SURF Descriptor

Now we have: Keypoint location + Scale + Orientation

But we still need to answer: What does the region around this keypoint actually look like

That's the job of the: **SURF Descriptor**

SURF takes a region around a keypoint and divides that region into smaller blocks(subregions).

The classic SURF descriptor uses: $4 \times 4 = 16$ subregions

Now For every subregion, SURF calculates Haar wavelet responses.

- d_x = response in x direction
- d_y = response in y direction

Then it calculates: $|d_x|$ and $|d_y|$

Now it summarizes the Haar responses in each subregion using 4 numbers

$$\boxed{\sum dx, \quad \sum dy, \quad \sum |dx|, \quad \sum |dy|}$$

Therefore: 1 subregion produces 4 values as their descriptor.

Hence the standard SURF descriptor is: **64-dimensional**.

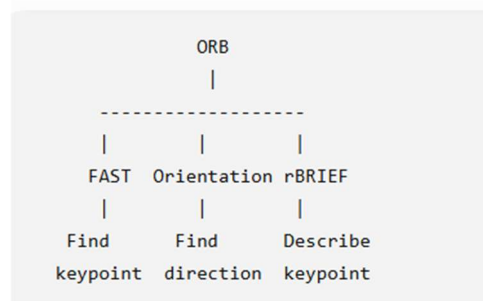
```
# SURF requires: pip install opencv-contrib-python
# and OPENCV_ENABLE_NONFREE build flag (often unavailable in prebuilt wheels)
surf = cv2.xfeatures2d.SURF_create(hessianThreshold=400)
keypoints, descriptors = surf.detectAndCompute(gray, None)
```

SURF is frequently unavailable in pip-installed OpenCV because of its historical patent/licensing considerations and is associated with the opencv-contrib ecosystem.

Know the theory for exams/interviews, but default to SIFT or ORB for real projects.

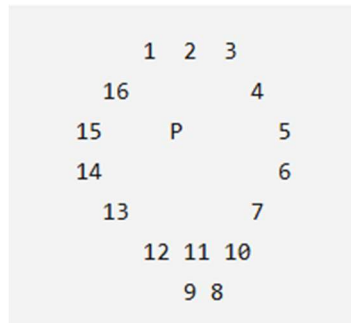
ORB (Oriented FAST and Rotated BRIEF)

- ORB is a **feature detection + feature description** technique used in computer vision.
- ORB finds interesting points in an image and creates a compact numerical description for each point so that the same points can be recognized in another image.
- ORB is free, open-source, and extremely fast — designed as a practical replacement for SIFT/SURF. It combines:
 - FAST keypoint detector (compares a circle of 16 pixels around a candidate to find corners quickly).
 - An orientation component computed from the intensity centroid, making FAST rotation-aware.
 - rBRIEF — a rotation-aware binary descriptor (256 bits) built from pairwise intensity comparisons.



Step 1: Keypoint Detection using FAST (Features from Accelerated Segment Test)

FAST is primarily a **corner detector** technique but it actually considers **16 pixels arranged around P** on a circle.



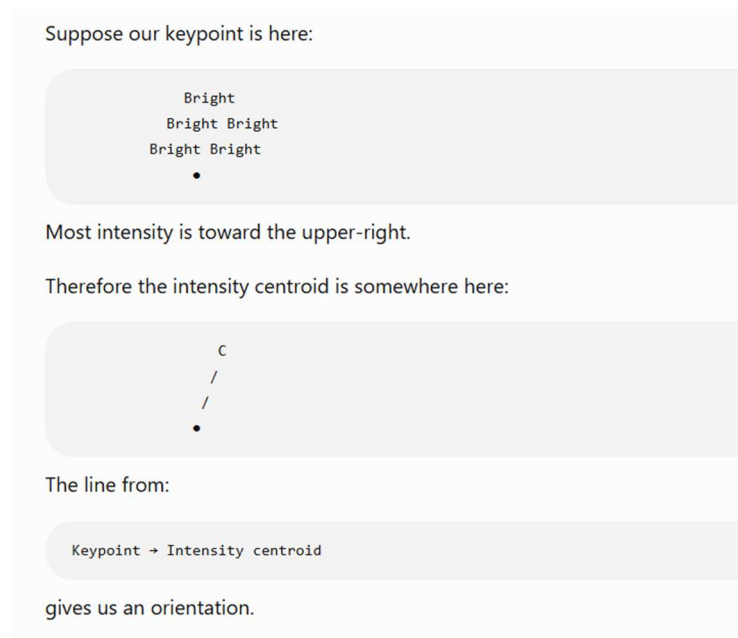
It checks whether a set of surrounding pixels is significantly brighter or darker than the centre.

- If enough surrounding pixels are significantly brighter or darker than the center, → the point(P) may be a corner.
- The commonly described FAST-9 test, for example, requires **at least 9 contiguous pixels** out of 16 satisfying the condition so that point P is said to be a corner.

Step 2: Orientation assignment for each detected keypoint using the intensity centroid.

- ORB looks at the pixels around the keypoint and asks: **In which direction is the brightness concentrated?**
- That gives us a direction and this is called the **Intensity Centroid**.
- Bright pixels contribute more.
- Dark pixels contribute less.
- ORB calculates where the "center of brightness" lies relative to the keypoint.

Example:



Mathematical idea behind intensity centroid

For a patch around the keypoint, ORB calculates image moments.

The basic moments are:

$$m_{00} = \sum I(x, y)$$

$$m_{10} = \sum xI(x, y)$$

$$m_{01} = \sum yI(x, y)$$

Then the centroid is:

$$C_x = \frac{m_{10}}{m_{00}}$$

$$C_y = \frac{m_{01}}{m_{00}}$$

And the orientation is approximately:

$$\theta = \tan^{-1} \left(\frac{C_y}{C_x} \right)$$

Step 3: Building Feature Descriptor using rBRIEF

- We take a small region around the keypoint now BRIEF(Binary Robust Independent Elementary Features) randomly chooses two points A and B and compare their value(intensity).
- Then it asks: Is pixel A brighter than pixel B?
- If $A > B \rightarrow$ Stores 1 otherwise stores 0

Above step is repeated 256 times and finally we get a binary descriptor

So, BRIEF converts the image patch into a **binary descriptor**.

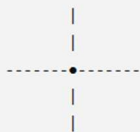
Here we are using binary descriptor which makes ORB:

- fast
- memory efficient
- suitable for real-time applications.

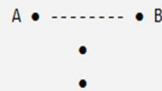
The problem with normal BRIEF is: **BRIEF doesn't handle rotation well.**

Example

Suppose your keypoint looks like this:

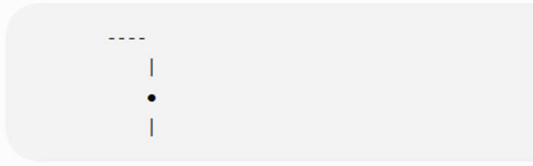


BRIEF has a predefined set of pixel pairs:



It compares those pairs.

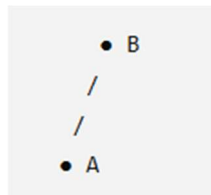
Now suppose the image rotates by 90°:



The **keypoint is still the same physical feature**, but the BRIEF sampling pattern hasn't rotated. So BRIEF may produce a completely different binary descriptor.

So that why we use **rBRIEF (rotated BRIEF)** to fix this

- ORB first finds the **orientation of the keypoint**.
- Let's say: orientation = 40°
- Then instead of using the BRIEF sampling pattern as it is: A • ----- • B
- ORB **rotates the sampling pattern by 40°** then perform the pixel comparison.



So ORB produces something like:

```
Keypoint 1
  Location = (120, 80)
  Orientation = 35°
  Descriptor = 101101001... [256 bits]

Keypoint 2
  Location = (250, 170)
  Orientation = 82°
  Descriptor = 010011010... [256 bits]
```

Because the descriptor is binary, matching uses fast Hamming-distance comparison instead of Euclidean distance

Hamming distance just compare the two binary representation and calculates the number of instances there are change in its representation.

For Example:

A = 101101

B = 100111

→ Hamming distance = 2

Practical: ORB in OpenCV

```
orb = cv2.ORB_create(nfeatures=500)
keypoints, descriptors = orb.detectAndCompute(gray, None)

print('Descriptor shape:', descriptors.shape)  # (N, 32) -> 32 bytes = 256 bits

out = cv2.drawKeypoints(img, keypoints, None, color=(0,255,0), flags=0)
show(cv2.cvtColor(out, cv2.COLOR_BGR2RGB), 'ORB Keypoints')
```

Matching Descriptors Between Two Images

Once descriptors are extracted from two images, matching finds correspondences — the core step for stitching, homography estimation, and recognition.

ORB → Binary descriptor

SIFT → Floating-point descriptor

Brute-Force Matching (ORB, binary descriptors)

Brute Force matching means comparing each descriptor from Image 1 against descriptors from Image 2.

```
img1 = cv2.imread('img1.jpg', cv2.IMREAD_GRAYSCALE)
img2 = cv2.imread('img2.jpg', cv2.IMREAD_GRAYSCALE)

orb = cv2.ORB_create()
kp1, des1 = orb.detectAndCompute(img1, None)
kp2, des2 = orb.detectAndCompute(img2, None)

bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True) # to remove some incorrect matches.
matches = bf.match(des1, des2)
matches = sorted(matches, key=lambda m: m.distance)  # best matches first

result = cv2.drawMatches(img1, kp1, img2, kp2, matches[:30], None,
                        flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
show(result, 'ORB Matches')
```

FLANN Matching (SIFT, float descriptors) + Ratio Test

FLANN stands for: **F**ast **L**ibrary for **A**pproximate **N**earest **N**eighbors

Using BF Match will be computationally expensive that's why we are using FLANN Matcher to match the descriptors.

Here we want to find which descriptor in Image 2 is closest to this descriptor in Image 1.

That's a **nearest-neighbor search** problem.

```
sift = cv2.SIFT_create()
kp1, des1 = sift.detectAndCompute(img1, None)
kp2, des2 = sift.detectAndCompute(img2, None)

index_params = dict(algorithm=1, trees=5) # FLANN_INDEX_KDTREE
search_params = dict(checks=50)
flann = cv2.FlannBasedMatcher(index_params, search_params)

# For each descriptor in Image 1, give me the two nearest descriptors from Image 2.
matches = flann.knnMatch(des1, des2, k=2)

# Lowe's ratio test – keep a match only if it's clearly better than the 2nd best
good = [m for m, n in matches if m.distance < 0.75 * n.distance]
print(f'{len(good)} good matches out of {len(matches)}')
```